Layer 4 vs Layer 7 Load Balancing

addresses and ports, or the whole HTTP request

Phase 3 videos on YouTube

In one line: a layer 4 load balancer sees only IP addresses, ports and TCP/UDP, so it is fast and works with any protocol; a layer 7 load balancer reads each HTTP request, so it can route by path, host or header, terminate TLS and balance individual HTTP/2 and gRPC requests, at more cost per request.

1 Two load balancers, two layers

Both are called load balancers, but they work at different layers of the network stack (Phase 2, lessons 5 to 7). Layer 4 is the **transport** layer: TCP and UDP, with ports. Layer 7 is the **application** layer: HTTP, with paths, headers and cookies. What a load balancer can see decides what it can do.

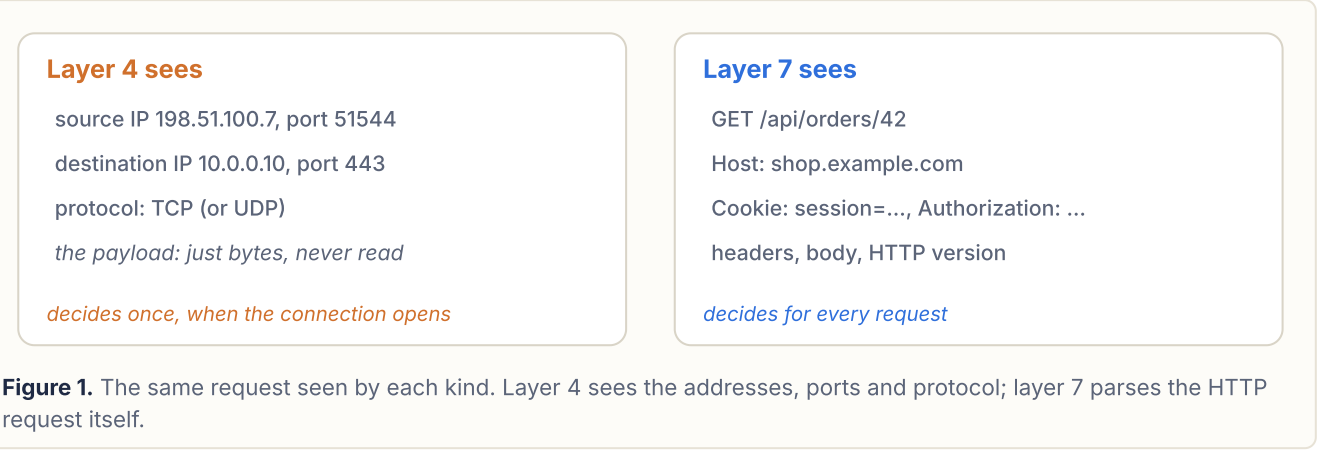


Figure 1. The same request seen by each kind. Layer 4 sees the addresses, ports and protocol; layer 7 parses the HTTP request itself.

2 Layer 4: fast and protocol-blind

A layer 4 load balancer picks a backend when a **connection opens** (using the source and destination IP and port), then just forwards the packets for that connection. It never parses the payload, so:

- it's very fast, adds little latency and handles millions of connections;
- it works with **any protocol** over TCP or UDP: databases, game servers, MQTT, custom binary protocols;
- it can pass encrypted TLS straight through without reading it.

The cost: it is blind to content. It can't route `/api/orders` differently from `/api/users`, and one long connection stays on one backend for its whole life.

3 Layer 7: reads every request

A layer 7 load balancer **terminates** the client's connection, reads the HTTP request and decides **per request**. That allows:

- routing by **path** (`/api/orders/*`) or **host name** (`api.example.com`);
- matching **headers or cookies** (send beta users to version 2);
- rewriting URLs and adding headers such as `X-Forwarded-For` (the client's real IP);

- **retrying** a failed request on another backend, and rate limiting.

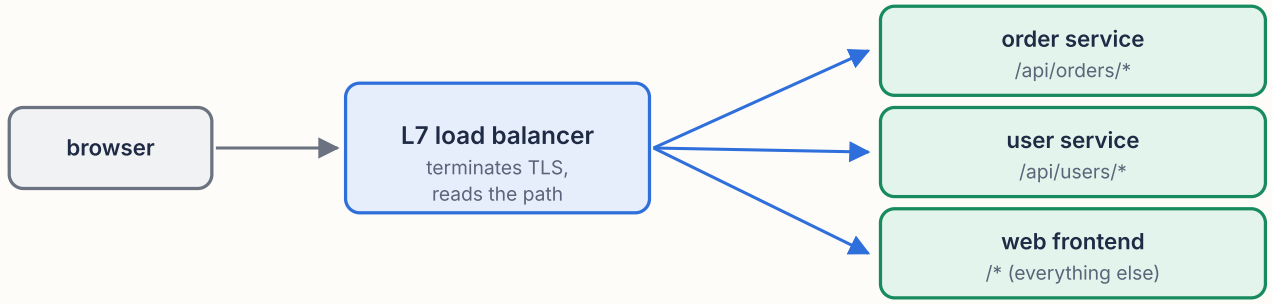


Figure 2. Path-based routing on one domain: the layer 7 load balancer reads the path and sends each request to the right service.

4 TLS: terminate or pass through

To read HTTP, a layer 7 load balancer must decrypt it, so it usually does **TLS termination**: it holds the certificate, decrypts, and sends plain HTTP (or a fresh internal TLS connection, called re-encryption) to the backends. Certificates live in one place, and the balancer can inspect requests.

With layer 4 you can do **TLS passthrough** instead: the encrypted bytes go straight to the backend, and only the backend ever decrypts them. Some compliance rules (end-to-end encryption, mutual TLS to the application) require that.

5 The trade-off, and the HTTP/2 trap

	Layer 4	Layer 7
Decides	once per connection	once per request
Speed and CPU	lowest latency, least CPU	more work per request
Protocols	anything over TCP/UDP	must understand it (HTTP, gRPC, WebSocket)
Routing	IP and port only	path, host, headers, cookies
TLS	pass through (or terminate)	terminates
AWS example	Network Load Balancer (NLB)	Application Load Balancer (ALB)

The trap: **HTTP/2 and gRPC** send many requests over **one long connection**. A layer 4 balancer pins that connection to one backend, so one backend gets every request while the others sit idle. You need layer 7 (or client-side balancing) to spread the individual requests.

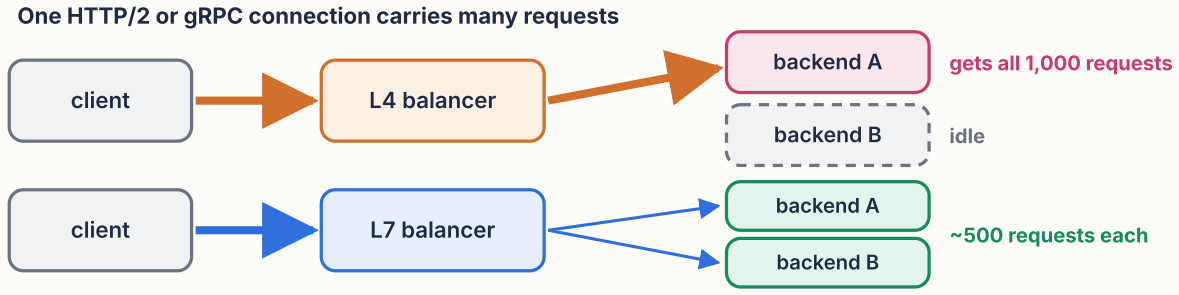


Figure 3. With one HTTP/2 connection, a layer 4 balancer sends all its requests to one backend; a layer 7 balancer spreads them.

WORKED EXAMPLE: A GAME AND A WEB PLATFORM

A multiplayer game uses a custom binary protocol over raw TCP. An AWS **Network Load Balancer** (layer 4) forwards those connections with very low latency, without needing to understand the protocol. Meanwhile, the company's web platform uses an **Application Load Balancer** (layer 7) to route `/api/orders` to the order service and `/api/users` to the user service, all on one domain and one certificate.

6 Using both

Many large systems stack them. A layer 4 balancer at the edge spreads millions of raw connections across a fleet of layer 7 proxies such as Envoy or NGINX, which then do the HTTP routing. The layer 4 tier is cheap and scales to huge connection counts; the layer 7 tier adds the smarts. Inside the cluster, a **service mesh** brings the same layer 7 features (retries, per-request balancing, mutual TLS) between services.

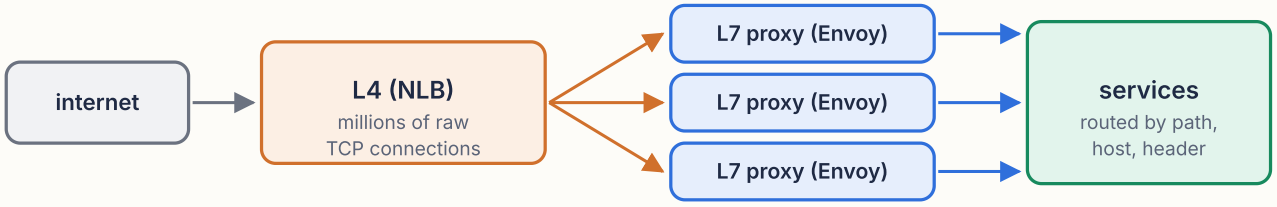


Figure 4. A common pattern: layer 4 at the edge for raw connection volume, layer 7 proxies behind it for HTTP routing.

COMMON MISTAKES

- Putting gRPC services behind a layer 4 balancer and wondering why one pod is at 100% CPU.
- Choosing layer 7 for a non-HTTP protocol it can't parse.
- Forgetting that after TLS termination the backend sees the balancer's IP, not the client's: read `X-Forwarded-For` (or use the PROXY protocol at layer 4).

7 Key terms

Layer 4 (transport): TCP/UDP, IP addresses and ports

TLS termination: the balancer decrypts and holds the certificate

Path routing: pick a service by URL path

NLB / ALB: AWS layer 4 / layer 7 load balancers

Layer 7 (application): HTTP: paths, headers, cookies

TLS passthrough: encrypted bytes go to the backend untouched

X-Forwarded-For: header carrying the original client IP

Multiplexing: many requests sharing one connection (HTTP/2)

8 Interview questions

Q1. When would you use a layer 4 instead of a layer 7 load balancer? BASIC

For non-HTTP protocols such as raw TCP or UDP, when you need the lowest latency and the highest connection counts, or when TLS must pass through to the backend untouched.

Q2. When do you need layer 7? BASIC

For routing by path, host or header, for TLS termination with one place for certificates, for retries and rate limits, and to balance individual HTTP/2 or gRPC requests that share one connection.

Q3. Your gRPC service has 10 pods, but one is at 100% CPU and the rest are idle. Why? SCENARIO

gRPC uses long-lived HTTP/2 connections, and a layer 4 balancer pins each connection to one pod. Use a layer 7 proxy (Envoy, a mesh) or client-side load balancing so each request is balanced, and cap connection age so clients reconnect.

Q4. Security requires that only the application ever sees decrypted traffic. What do you use? SCENARIO

A layer 4 load balancer with TLS passthrough, so the backend terminates TLS. You lose path routing at the balancer, so route by port or SNI, or do routing in the application tier.

Q5. After moving behind an ALB, every request in your logs comes from the same few IPs. Why? TRICKY

The ALB terminates the connection and opens its own to the backend, so the source IP is the ALB's. The real client IP is in the X-Forwarded-For header; log that, and trust it only from your balancer.

Q6. Why not just use layer 7 everywhere? TRICKY

It costs CPU and latency per request, must understand each protocol, and terminates TLS. For huge connection counts, non-HTTP traffic or passthrough needs, layer 4 is simpler and cheaper; many systems put layer 4 in front of layer 7.

Next: SD-G-003 · Round Robin and Weighted Round Robin backendroadmap.com